

Open-Source Verification with CocoTB

SyoSil, 2026

Contents

- Introduction to CocoTB
- Basic testbench in CocoTB
- Coroutines
- Using coroutines in a testbench
- Advanced CocoTB concepts
- Putting it all together

Introduction to CocoTB

Introduction to CocoTB

- **CocoTB** is an open-source cosimulation framework in Python
- Cosimulation: CocoTB is an **interface to the simulator** in Python
 - CocoTB is *not* a hardware description language
 - CocoTB is *not* a simulator
 - CocoTB allows you to interface the simulator directly from python
 - CocoTB allows for test creation

Why use CocoTB?

- Alternative to writing testbenches in VHDL or SystemVerilog
- Allows you to benefit from thousands of Python packages
- Easier to have a “software” mindset when verifying hardware

Requirements

- Python installation (minimum version 3,6+)
- CocoTB Python module
- A hardware simulator
- That's it!

Basic Testbenches with CocoTB

CocoTB Test Structure

- CocoTB tests are python functions decorated with `@cocotb.test()`
- CocoTB tests are *coroutines*, so they must be declared as *async* functions
- CocoTB tests are run using a Makefile
- More info: <https://www.cocotb.org>
- Docs: <https://docs.cocotb.org/en/v2.0.1/>

Hello world in RTL: Blink

The module

```
module blink (  
    input logic clk,  
    input logic reset,  
    output logic led  
);  
  
logic [1:0] cnt, cnt_next;  
logic led_state, led_state_next;  
  
always_comb begin  
    cnt_next = cnt + 2'd1;  
    led_state_next = (cnt == '3) ?  
                     !led_state : led_state;  
end  
  
always_ff @(posedge clk) begin  
    if (reset) begin  
        cnt <= '0;  
        led_state <= '0;  
    end else begin  
        cnt <= cnt_next;  
        led_state <= led_state_next;  
    end  
end  
  
assign led = led_state;  
endmodule
```

The testbench

```
import cocotb  
from cocotb.triggers import Timer  
  
@cocotb.test()  
async def my_first_test(dut):  
  
    # Reset the registers  
    dut.reset.value = 1  
    dut.clk.value = 0  
    await Timer(1, units="ns")  
    dut.clk.value = 1  
    await Timer(1, units="ns")  
    dut.reset.value = 0  
  
    # Let the clock go on  
    for _ in range(20):  
        dut.clk.value = 0  
        await Timer(1, units="ns")  
        dut.clk.value = 1  
        await Timer(1, units="ns")
```

Takeaways from Python testbench

Takeaways from first Python testbench

- Tests are annotated with `@cocotb.test()`
- Test function are `async`
- Test function has argument **dut** that allows us to access the DUT and its signals
- Signals are accessed through **dut.<name>.<value>**
 - This is because signals are objects.
Allows you to pass signal handles between functions in your TB

Makefiles

Makefiles are used to run tests

- make is a build-management system.
 - Tracks which files are changed, helps make development easier
- Argument COCOTB_TOPLEVEL defines the RTL module to simulate
- Argument COCOTB_TEST_MODULES defines the Python testbenches to use
- Can add WAVES=1 to dump a waveform file
- To run: make COCOTB_TEST_MODULES =__

CocoTB Testbench Structure

- CocoTB has a several quality-of-life features that make testbench development faster
 - Automatic clock generation
 - Triggers that allow you to await a certain condition
 - Coroutines for parallel execution

Better Test Benches: Automatic Clocking

- We don't want to deal with manually toggling the clock.
- Instead, we use a CocoTB coroutine to handle it for us
 - `Clock(signal, period, units)` generates a clock object
 - `clk.start()` starts the clock
- New testbench structure on the right:

```
import cocotb
from cocotb.triggers import Timer
from cocotb.clock import Clock

@cocotb.test()
async def my_second_test(dut):
    # Create a clocking coroutine
    clk = Clock(signal=dut.clk, period=2,
                units="ns")
    clk.start()

    # Reset the registers
    dut.reset.value = 1
    await ClockCycles(dut.clk, 1)
    dut.reset.value = 0
    await ClockCycles(dut.clk, 20)
```

Coroutines in CocoTB

Coroutines in CocoTB

- Coroutines: Functions that process in parallel
 - `cocotb.Clock(...)` creates a clock object, and `clk.start()` starts the clock
- Not limited to included coroutines. Can create coroutines for any task
 - Toggling reset and starting clock
 - Driving inputs onto the DUT
 - Checking outputs on the DUT
 - Monitoring control interface to DUT
 - ...

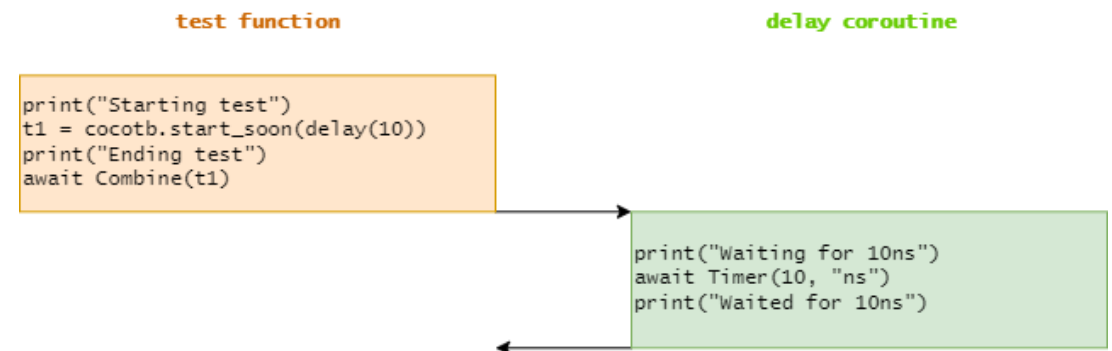
Coroutines in CocoTB

- Coroutines are functions that execute in parallel
- All coroutines are declared as `async` functions
- Coroutine execution is handled by the scheduler
 - Coroutines will execute until they *yield control* to another coroutine
- Coroutines yield control when they're waiting for something to happen
 - Indicated by the `await` keyword
 - Time passing, clock edge, number of signal transitions, ...
- Coroutines are started with `cocotb.start_soon(coro)`
 - We say that we *fork off* a new thread

Using cocotb.start_soon

- Cocotb.start_soon(...) schedules a coroutine but does not yield control
- Should not be prepended with *await*
 - Await keyword performs the yielding of control
- May need to use *combine* keyword
- Result:
 - [0ns] Before delay
 - [0ns] After delay
 - [0ns] Waiting for 10ns
 - [10ns] Waited for 10ns

```
async def delay(N):  
    print(f"Waiting for {N}ns")  
    await Timer(N, "ns")  
    print(f"Waited for {N}ns")  
  
@cocotb.test()  
async def using_start_soon(dut):  
    print("Starting test")  
    t1 = cocotb.start_soon(delay(10))  
    print("Ending test")  
    await Combine(t1)
```



Not scheduling coroutines

- If we don't schedule a coroutine with `cocotb.start_soon(...)`, the coroutine will execute without yielding control.

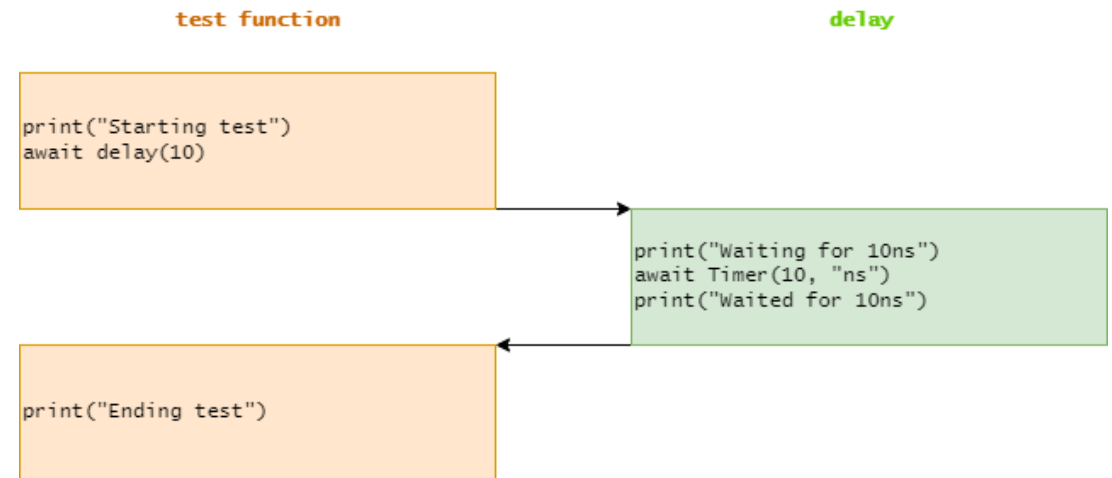
```
async def delay(N):  
    print(f"Waiting for {N}ns")  
    await Timer(N, "ns")  
    print(f"Waited for {N}ns")
```

```
@cocotb.test()  
async def using_no_start(dut):  
    print("Before delay")  
    await delay(10)  
    print("After delay")
```

- No need to use `combine` keyword (no coroutines to `await`)

- Result:

```
[0ns] Before delay  
[0ns] Waiting for 10ns  
[10ns] Waited for 10ns  
[10ns] After delay
```



Complex Scheduling

- What do you think will happen in this scenario?

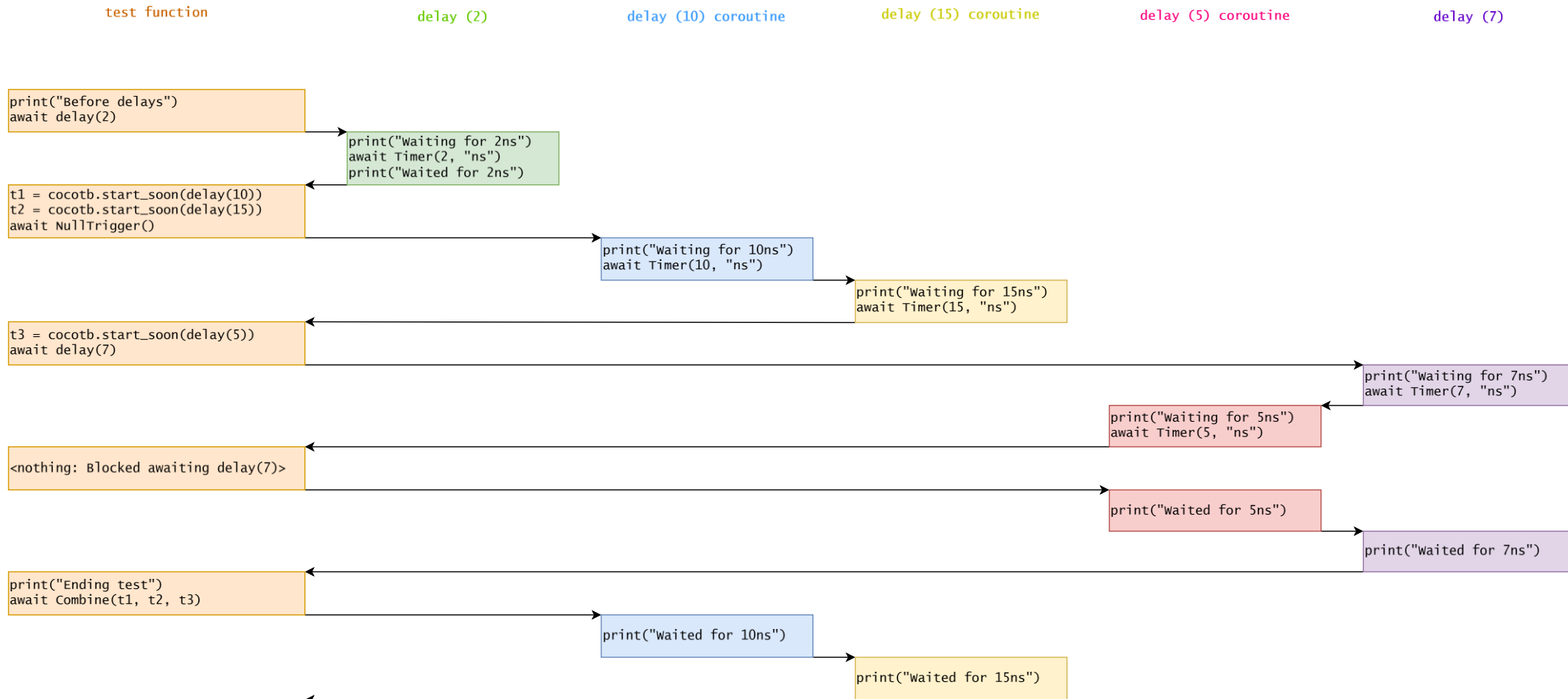
- Result

```
[0ns] Before delays
[0ns] Waiting for 2ns
[2ns] Waited for 2ns
[2ns] Waiting for 10ns
[2ns] Waiting for 15ns
[2ns] Waiting for 7ns
[2ns] Waiting for 5ns
[7ns] Waited for 5ns
[9ns] Waited for 7ns
[9ns] After delays
[12ns] Waited for 10ns
[17ns] Waited for 15ns
```

```
async def delay(N):
    print(f"Waiting for {N}ns")
    await Timer(N, "ns")
    print(f"Waited for {N}ns")

@cocotb.test()
async def fork_multiple(dut):
    print(f"Before delays")
    await delay(2)
    t1 = cocotb.start_soon(delay(10))
    t2 = cocotb.start_soon(delay(15))
    await NullTrigger()
    t3 = cocotb.start_soon(delay(5))
    await delay(7)
    print(f"After delays")
    await Combine(t1, t2, t3)
```

Complex scheduling



Complex Scheduling

Important takeaways

- Initial `await delay(2)` statement is not a coroutine, so it blocks execution of the rest of the testbench
- When `await Timer(N, "ns")` is encountered, the first coroutine to start is `delay(10)`
 - Because it was added to the scheduler before `delay(15)`
 - Not a certainty! Do not depend on coroutines in the same timestep executing in a specified order
- Last `await delay(7)` is not a coroutine, so it blocks execution of the next line
 - Moves into `delay(7)` first, but then yields and starts the `delay(5)` coroutine
 - Coroutines can still execute while blocked on `delay(7)`
 - Notice that `delay(5)` finishes while waiting for `delay(7)`

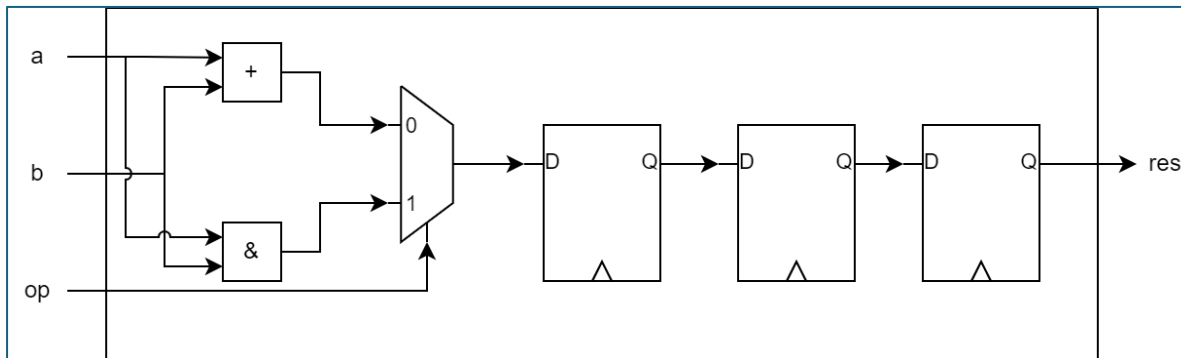
```
async def delay(N):
    print(f"Waiting for {N}ns")
    await Timer(N, "ns")
    print(f"Waited for {N}ns")

@cocotb.test()
async def fork_multiple(dut):
    print(f"Before delays")
    await delay(2)
    t1 = cocotb.start_soon(delay(10))
    t2 = cocotb.start_soon(delay(15))
    await NullTrigger()
    t3 = cocotb.start_soon(delay(5))
    await delay(7)
    print(f"After delays")
    await Combine(t1, t2, t3)
```

Using Coroutines in Testbenches

Using Coroutines in Testbenches

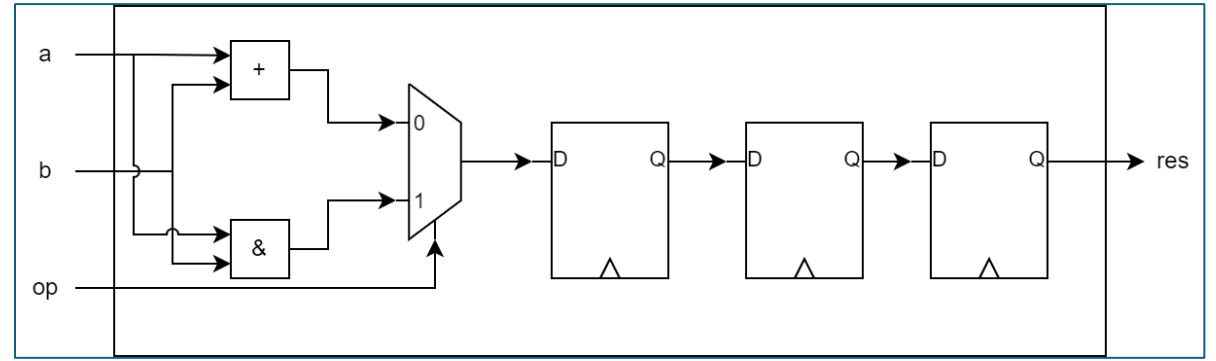
- DUT is a very simple ALU.
 - Can ADD or AND two values.
- Pipeline is three clock cycles deep



```
module alu (  
    input clk, reset,  
  
    input logic [7:0] a, b,  
    input logic op,  
    output logic [7:0] res  
);  
logic [7:0] pipe_0, pipe_1, pipe_2;  
logic [7:0] alu_res;  
  
always_comb begin  
    if (op) begin  
        alu_res = a & b;  
    end else begin  
        alu_res = a + b;  
    end  
end  
  
always_ff @(posedge clk) begin  
    if (reset) begin  
        pipe_0 <= '0;  
        pipe_1 <= '0;  
        pipe_2 <= '0;  
    end else begin  
        pipe_0 <= alu_res;  
        pipe_1 <= pipe_0;  
        pipe_2 <= pipe_1;  
    end  
end  
  
assign res = pipe_2;  
endmodule: alu
```

Using Coroutines in Testbenches

- How to verify this? Cannot easily drive inputs and check same outputs in one function
- Don't want to inspect waveforms. Test should be self-checking
- Solution: Use coroutines



Driving Inputs

- Coroutine for driving inputs
 - When driving inputs, be aware of race conditions if signals are driven conditionally on the same clock edge as the DUT.
 - Do not drive signals asynchronous to the clock i.e. never `await` any DUT signals that are not the clock!

```
# Drive inputs onto the DUV
async def drive_inputs(dut, a, b, op):
    for a_item, b_item, op_item in zip(a, b, op):
        await RisingEdge(dut.clk)
        dut.a.value = a_item
        dut.b.value = b_item
        dut.op.value = op_item
```

Checking Outputs

- Coroutine for checking outputs
 - Start at same time as input driver, so include a 3-cycle delay
 - Use Python keyword **assert** to check for correctness
 - Do not forget to wait for the end of the current time step
 - Pass a string as second parameter that will be shown if error occurs
 - No asynchronous reads. Do not **await** any signal but the clock!

```
# Check results from DUV
async def check_outputs(dut, res):
    # We know there is a 3-cycle delay.
    # Wait three cycles before checking
    await ClockCycles(dut.clk, 3)

    for res_item in res:
        await RisingEdge(dut.clk)
        await ReadOnly()
        assert dut.res.value == res, f"{i} {dut.res.value} != {res}"
```

Setting up Clock and Reset

- Function for setting up clock and reset signals
 - Must also give all inputs an initial value
- Function to generate randomized inputs and correct outputs

```
# Generate a clock signal, toggle reset, tie inputs low
async def setup_clk_reset(dut):
    # Setup clock
    Clock(dut.clk, 2, "ns").start()
    dut.a.value = 0
    dut.b.value = 0
    dut.op.value = 0
    dut.reset.value = 1
    await ClockCycles(dut.clk, 2)
    dut.reset.value = 0

# Generate randomized inputs and output vectors
def gen_input_output():
    (a, b, op, res) = ([], [], [], []) # Create empty lists
    for i in range(50):
        a.append(random.randint(0,255))
        b.append(random.randint(0,255))
        op.append(random.randint(0,1))
        if (op[i] == 0):
            res.append((a[i] + b[i]) & 0xFF)
        else:
            res.append(a[i] & b[i])
    return (a, b, op, res)
```

Testbench Structure

- Not using start or start_soon to launch setup_clk_reset
 - Ensures that reset has been toggled on and off before returning from function
- Using start_soon to run drive_inputs and check_outputs in parallel
- Awaiting Combine(...) trigger so test doesn't stop before both coroutines return

```
import cocotb
import random
from cocotb.triggers import *
from cocotb.clock import Clock

async def setup_clk_reset(dut):
    ...

async def gen_input_output():
    ...

async def drive_inputs(dut, a, b, op):
    ...

async def check_outputs(dut, res):
    ...

@cocotb.test()
async def test_alu(dut):
    (a, b, op, res) = gen_input_output()
    await setup_clk_reset(dut)
    task_drive = cocotb.start_soon(drive_inputs(dut, a, b, op))
    task_check = cocotb.start_soon(check_outputs(dut, res))
    await Combine(task_drive, task_check)
```

Advanced CocoTB Concepts

Triggers

- Triggers are used to wait for specific events.
- Coroutines `await` a trigger, pausing execution until the trigger occurs

Triggers

- Edge triggers
 - Edge, RisingEdge, FallingEdge
 - RisingEdge, FallingEdge only well-defined for 1-bit signals
 - Edge can also be used to detect changes on multi-bit signals
- Clock cycle counter
 - ClockCycles(signal, N)
 - Fires after N transitions of signal
- Timing triggers
 - Timer(time, units)
 - Fires after the given amount of simulation time units has passed

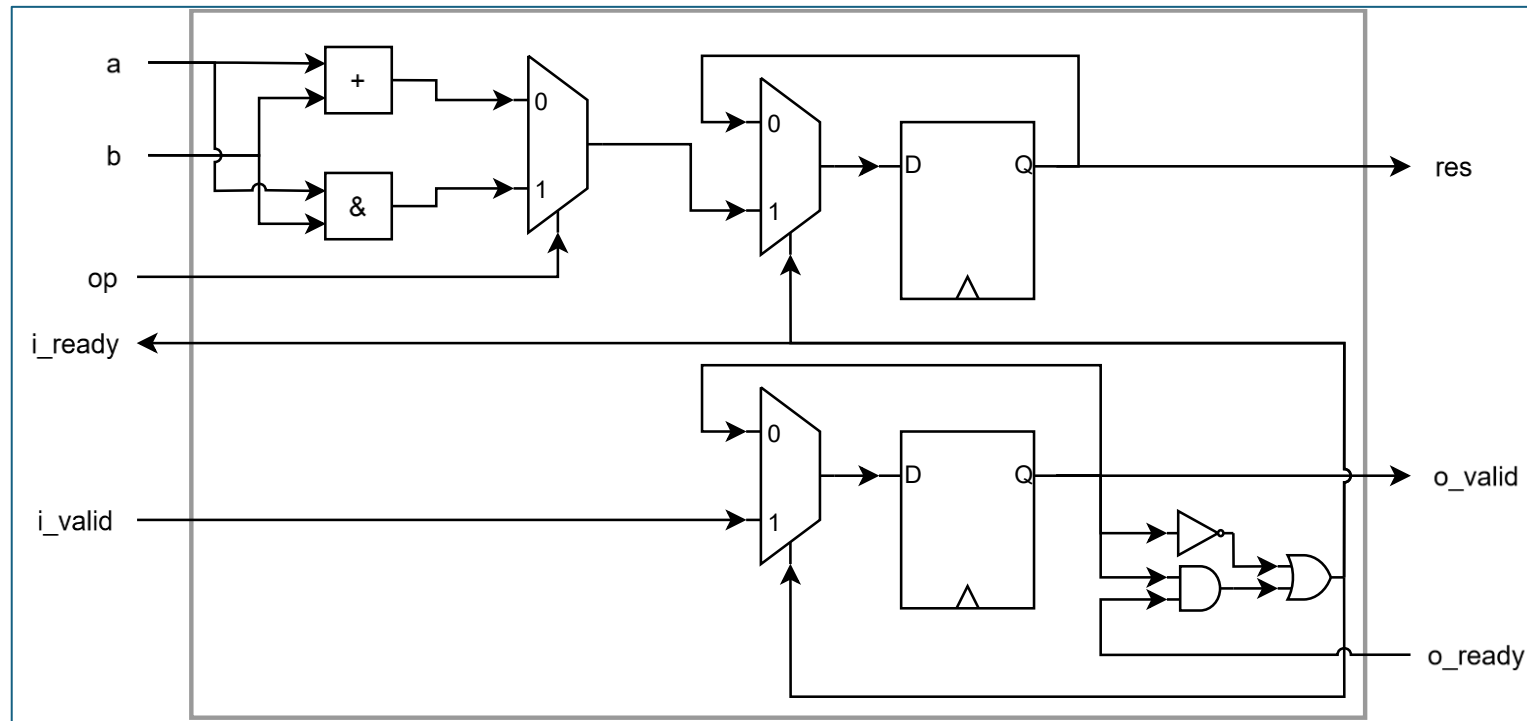
Synchronization triggers

- Combine
 - `Combine(t1, t2, ..., tN)`
 - Fires when all the given triggers have fired
- First
 - `First(t1, t2, ..., tN)`
 - Fires when the first given trigger fires

Putting it All Together

New ALU Design

- ALU now uses ready/valid signaling instead of 3-deep pipeline
- How does this change our testbench?



New Coroutines

- Need coroutines for driving inputs and sampling outputs that also respect the ready/valid protocol
- For proper verification, also need to randomize the delay values

Driving Inputs

- For each input, wait 0-3 clock cycles before making valid
- Once valid, wait until input-ready, indicating that data has been sampled. Then pull valid low again
- `i_ready` is combinationally tied to `o_ready`
- Good testbench principle
 - Drive and sample rising clock edge

```
# Helper function to await signal value
async def is_level(clk, signal, value):
    await RisingEdge(clk)
    await ReadOnly()
    while(signal.value != value):
        await RisingEdge(clk)
        await ReadOnly()

async def drive_inputs(dut, a, b, op):
    for a_item, b_item, op_item in zip(a, b, op):
        delay = random.randint(0, 3)
        await ClockCycles(dut.clk, delay)

        # Push values
        dut.i_valid.value = 1
        dut.a.value = a_item
        dut.b.value = b_item
        dut.op.value = op_item

        # Wait for i_ready to be 1
        # Note: Not using .value here.
        #       Passing object references and not signal values
        await is_level(dut.clk, dut.i_ready, 1)

        # Drive i_valid low on falling clock edge
        await RisingEdge(dut.clk)
        dut.i_valid.value = 0
```

Receiving outputs

- For each result, wait 0-5 clock cycles before marking ready
- Once ready, wait until valid, indicating that data has been sampled. Then pull ready low again
- `o_valid` is not dependent on signals but is driven by register
 - Can still use `is_level` to check since rising edge of clock comes before signal updates

```
async def check_outputs(dut, res):
    for res_item in res:
        # Wait before taking o_ready high
        delay = random.randint(0, 5)
        await ClockCycles(dut.clk, delay)

        # Drive on rising clock edge
        dut.o_ready.value = 1

        # Check on rising clock edge
        await is_level(dut.clk, dut.o_valid, 1)
        assert dut.res.value == res_item

        # Drive on rising clock edge
        await RisingEdge(dut.clk)
        dut.o_ready.value = 0
```

Testbench Structure

- Exact same testbench structure as previous example

```
import cocotb
import random
from cocotb.triggers import *
from cocotb.clock import Clock

async def setup_clk_reset(dut):
    ...

async def gen_input_output():
    ...

async def is_level(clk, signal, value):
    ...

async def drive_inputs(dut, a, b, op):
    ...

async def check_outputs(dut, res):
    ...

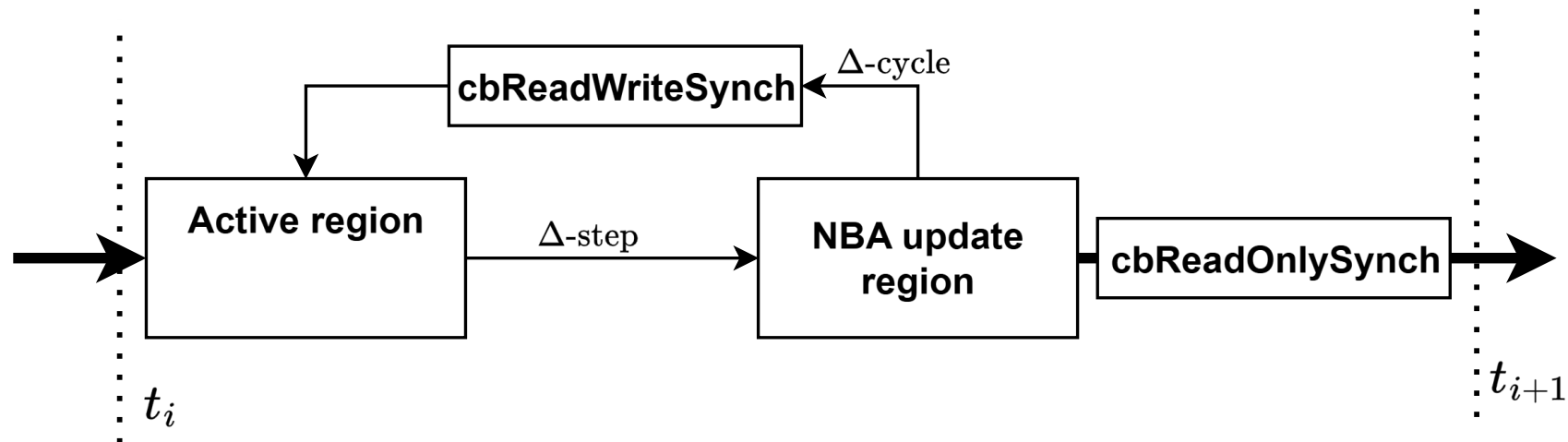
@cocotb.test()
async def test_alu(dut):
    (a, b, op, res) = gen_input_output()
    await setup_clk_reset(dut)
    task_drive = cocotb.start_soon(drive_inputs(dut, a, b, op))
    task_check = cocotb.start_soon(check_outputs(dut, res))
    await Combine(task_drive, task_check)
```

Key Take-Aways

- Coroutines are a powerful tool for decoupling complex tasks
- Use triggers to wait for specific events
- Drive and sample signals on rising clock edge

Connecting to the Simulator: VPI Scheduling

- VPI offers different callbacks for points in the simulation cycle
- Two are important to us:
 - `cbReadWriteSynch` (Technically the IEEE is ambiguous if this after the active or NBA active update region)
 - `cbReadOnlySynch`



CocoTB Scheduler

- CocoTB interfaces VPI

ReadWriteSynch

Syntax:

```
await ReadWrite()
```

ReadOnlySynch

Syntax:

```
await ReadOnly()
```

Race Conditions

- Testbench also runs in simulation cycle

testbench:

```
<reset>
await RisingEdge(dut.clk)
dut.d.value = 1
print(dut.q.value)
```

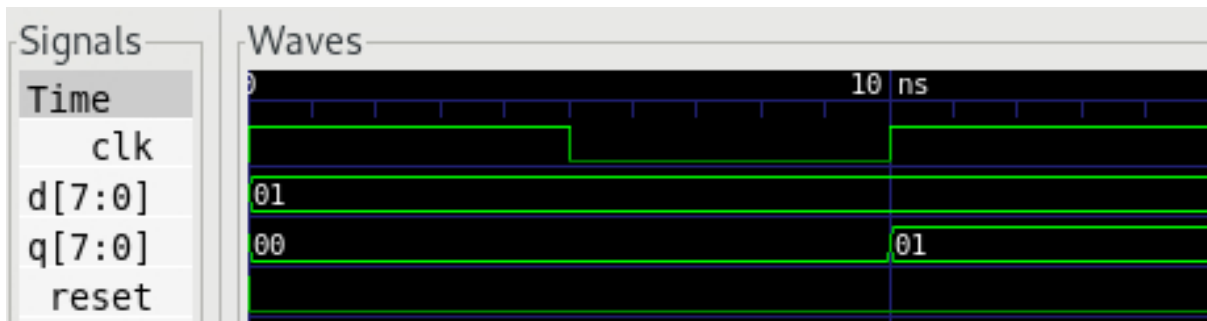
RTL:

```
module dff (
    input clk, reset,
    input logic [7:0] d,
    output logic [7:0] q
);

always_ff @(posedge clk) begin
    if (reset) begin
        q <= '0;
    end else begin
        q <= d;
    end
end

endmodule: dff
```

Waveform



Race Conditions

- Testbench also runs in simulation cycle

testbench:

```
<reset>
await RisingEdge(dut.clk)
dut.d.value = 1
print(dut.q.value)
await ReadOnly()
print(dut.q.value)
```

RTL:

```
module dff (
    input clk, reset,
    input logic [7:0] d,
    output logic [7:0] q
);

always_ff @(posedge clk) begin
    if (reset) begin
        q <= '0;
    end else begin
        q <= d;
    end
end

endmodule: dff
```

10.00ns	INFO	cocotb	q before ReadOnly(): 00000000
10.00ns	INFO	cocotb	q after ReadOnly(): 00000001